

Check This New Sorting Algorithm Design

In this article, which is aimed at those who love playing around with algorithms and data structures—programmers, developers and coders—the author shares a new sorting algorithm developed by him, which reduces sort time drastically. It assumes that the readers are aware about at least asymptotic analysis, worst and best case running time analysis and basic sorting algorithms.

Sorting algorithms, which arrange the elements of a list in a certain order (either ascending or descending), are an important category in computer science. We can use sorting as a technique to reduce search complexity. Vast amounts of research has gone into this category of algorithms because of its importance, particularly in database algorithms.

Classification

Sorting algorithms are generally classified into different categories based on various factors like the number of comparisons, memory usage, recursion, etc. Existing sorting algorithms include the *Selection*, *Bubble*, *Heap*, *Insertion*, *Quick*, *Merge*, *Counting*, *Bucket* and *Radix* sort. All these different algorithms have some advantages and disadvantages.

The description of each of these algorithms is beyond the scope of this article.

The following gives a glimpse of how sorting works (with the Bubble sort):

```
Void bubble_sort(int arr[], int n) {
    for(int pass=n-1;pass>=0;pass--)
        // outer loop executed n times
    {
        for( int i=0; i<pass-1;i++)
            // inner loop execute n times
        {
            if(arr[i]> arr[i+1])
                // swapping the elements
                {
                    int temp=arr[i];
                    arr[i]= arr[i+1];
                    arr[i+1]=temp;
                }
        }
    }
}
```



The above algorithm takes $O(n^2)$ time even in the best case.

Try optimising it so that we can skip some extra swaps by introducing an extra flag.

The following is optimised code of the above:

```
Void bubble_sort(int arr[], int n)
{
    int pass, i, temp, swapped=1;
    for( pass=n-1; pass>=0&& swapped; pass--)
    {
        Swapped=0;
        for( i=0; i<pass-1;i++)
        {
            if(arr[i]> arr[i+1])
            {
                temp=arr[i];
                arr[i]=arr[i+1];
                arr[i+1]=temp;
                swapped=1;
            }
        }
    }
}
```

This optimised version finally improves the time of the worst and best cases of the Bubble sort to $O(n)$. This is called an optimisation of the code.

My new sorting algorithm

Recently, I developed a sorting algorithm that is able to sort any input (i.e., not ask for input restrictions) in $O(n)$ time with $O(n)$ Space Complexity. Though there are several sorting algorithms out there, which I mentioned earlier, some like Merge, Quick and Heap sort take $O(n \log n)$ time, and some like Bubble, Selection and Insertion sort, take $O(n^2)$ time; but no algorithm takes $O(n)$ time to throw up the result.